UNIT-5

1. Introduction to GPT Models-Evolution from GPT-1 to GPT-3

GPT models are decoder-only Transformers, designed for autoregressive language modeling, where the goal is to predict the next token in a sequence given previous tokens. Unlike bidirectional models like BERT, GPT's unidirectional (left-to-right) approach makes it ideal for generative tasks, such as text completion, dialogue systems, and creative writing. The evolution from GPT-1 to GPT-3 reflects advancements in model scale, training data, and learning paradigms, enabling broader applications with minimal fine-tuning.

GPT-1: The Foundation (2018)

Architecture:
- Structure: 12-layer Transformer decoder with 117 million parameters.
- Self-Attention: Multi-head self-attention (12 heads, 768-dimensional embeddings) with masked attention to ensure autoregressive generation.
- Positional Encoding: Sinusoidal encodings to capture token order.

Pretraining:
- Objective: Causal Language Modeling (CLM), predicting the next token in a sequence.
- Dataset: BookCorpus (~800M words), containing 7,000 unpublished books, providing diverse narrative text.
- Training: Unsupervised, maximizing the likelihood of token sequences.

Fine-Tuning:
- Adapted for tasks like classification, question answering, and text entailment by adding task-specific layers and training on labeled datasets (e.g., SST-2 for sentiment analysis).
- Demonstrated transfer learning, outperforming traditional models with limited task-specific data.

Impact:
- Introduced the concept of pretraining a Transformer for general language understanding, followed by task-specific fine-tuning.
- Showed competitive performance on benchmarks like GLUE, though limited by scale and unidirectional context.

Limitations:
- Small dataset and model size constrained generalization.
- Unidirectional context less effective for tasks requiring full sequence understanding (e.g., question answering).

GPT-2: Scaling Up (2019)

Architecture:
- Structure: Scaled to multiple sizes (117M to 1.5B parameters), with the largest model having 48 layers, 1,600-dimensional embeddings, and 10,240 vocabulary size.
- Improvements: Larger context window (1,024 tokens vs. GPT-1's 512), layer normalization moved to the input of each sub-block for stability.

Pretraining:
- Objective: CLM, same as GPT-1.
- Dataset: WebText, a curated dataset of ~8M web pages (~40GB of text) scraped from Reddit links, offering diverse, high-quality content.
- Training: Emphasized zero-shot performance, reducing reliance on fine-tuning.

Advancements:
- Demonstrated strong zero-shot capabilities, generating coherent text without task-specific training.
- Improved text quality, handling longer sequences and maintaining coherence in creative writing.
- Introduced safety concerns, leading OpenAI to initially limit the release of the full model due to potential misuse (e.g., generating fake news).

Impact:
- Showcased the power of scaling model size and data, producing human-like text for applications like chatbots and story generation.
- Influenced research into large-scale language models, highlighting the trade-off between performance and ethical risks.

Limitations:
- Still unidirectional, limiting performance on understanding-heavy tasks.
- High computational cost for training and inference, inaccessible to smaller organizations.

GPT-3: The Paradigm Shift (2020)

Architecture:
- Structure: Massive scale with 175 billion parameters, 96 layers, 12,288-dimensional embeddings, and a 2,048-token context window.
- Design: Similar to GPT-2 but with sparse attention mechanisms (e.g., alternating dense and sparse layers) to improve efficiency.

Pretraining:
- Objective: CLM, predicting the next token.
- Dataset: Expanded to ~570GB of text, including Common Crawl (filtered for quality), WebText2, Books1, Books2, and Wikipedia, ensuring diverse, high-volume data.
- Training: Required significant computational resources (e.g., thousands of GPUs), emphasizing scale as a driver of performance.

Advancements:
- Few-Shot Learning: GPT-3 introduced zero-shot, one-shot, and few-shot learning, where the model performs tasks with minimal examples provided in the prompt (e.g., “Classify this review as positive or negative: ‘Great movie!’” → “Positive”).
- Broad Capabilities: Excelled in diverse tasks—text generation, translation, question answering, code generation—without fine-tuning.
- In-Context Learning: Leveraged large context windows to learn from prompts, reducing the need for gradient-based fine-tuning.

Impact:
- Redefined NLP applications, powering tools like chatbots (e.g., early versions of ChatGPT), code assistants, and content generation platforms.
- Demonstrated emergent abilities, such as basic reasoning and cross-lingual understanding, due to scale.

- Sparked discussions on AI ethics, including bias, energy consumption, and misuse risks (e.g., disinformation).

Limitations:
- Enormous computational and energy costs, limiting accessibility.
- Potential for biased or harmful outputs, requiring careful prompt engineering and moderation.
- Unidirectional context still less effective for tasks needing bidirectional understanding (e.g., sentiment analysis compared to BERT).

Evolution Summary

- Scale: GPT-1 (117M parameters) → GPT-2 (1.5B) → GPT-3 (175B), showing that larger models and datasets improve performance.
- Training Data: BookCorpus (800M words) → WebText (40GB) → Common Crawl + others (570GB), increasing diversity and quality.
- Capabilities: Fine-tuning (GPT-1) → zero-shot (GPT-2) → few-shot learning (GPT-3), reducing task-specific training needs.
- Applications: Expanded from classification and entailment to generation, reasoning, and cross-task generalization.
- Challenges: Computational cost, ethical concerns, and unidirectional limitations grew with scale.

Relevance to Sentiment Analysis

In the context of the sentiment analysis case study, GPT models offer an alternative to BERT:
- GPT-1: Requires fine-tuning for sentiment classification, similar to BERT, but less effective due to unidirectional context.
- GPT-2: Can perform zero-shot sentiment analysis with prompts (e.g., “Is this review positive or negative?”), though accuracy may lag behind BERT or RoBERTa.
- GPT-3: Excels in few-shot sentiment analysis, classifying reviews with minimal examples (e.g., “Positive: ‘Loved it!’ Negative: ‘Hated it!’ Review: ‘Amazing film!’ → Positive”). However, its generative focus makes it less optimized for classification compared to encoder-based models like BERT.

Implementation: Few-Shot Sentiment Analysis with GPT-2

Below is a TensorFlow implementation using a pretrained GPT-2 model (via the `transformers` library) for few-shot sentiment analysis, demonstrating its generative approach to classification.

```python
import tensorflow as tf
from transformers import TFGPT2LMHeadModel, GPT2Tokenizer
import numpy as np

# Initialize model and tokenizer
model_name = 'gpt2'
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = TFGPT2LMHeadModel.from_pretrained(model_name)

# Few-shot prompt for sentiment analysis
prompt_template = """
The following are examples of sentiment analysis:
Text: "This movie was fantastic, loved every minute!" Sentiment: Positive
```

```
Text: "Terrible film, such a waste of time." Sentiment: Negative
Text: "{}" Sentiment:"""

def predict_sentiment(text, model, tokenizer, max_len=512):
    # Create prompt
    prompt = prompt_template.format(text)

    # Tokenize input
    encoded = tokenizer(prompt, max_length=max_len, truncation=True, return_tensors='tf')
    input_ids = encoded['input_ids']
    attention_mask = encoded['attention_mask']

    # Generate output
    outputs = model.generate(
        input_ids=input_ids,
        attention_mask=attention_mask,
        max_new_tokens=10,
        num_return_sequences=1,
        pad_token_id=tokenizer.eos_token_id
    )

    # Decode output
    generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)

    # Extract sentiment
    sentiment = generated_text.split("Sentiment:")[-1].strip().split()[0]
    return sentiment

# Example usage
test_sentence = "This movie was absolutely fantastic, with stunning visuals!"
sentiment = predict_sentiment(test_sentence, model, tokenizer)
print(f"Text: {test_sentence}")
print(f"Sentiment: {sentiment}")
```

This code demonstrates few-shot sentiment analysis with GPT-2, using a prompt to guide the model to classify sentiment. For production, fine-tuning on a dataset like IMDB or using a larger model (e.g., GPT-3 via API) would improve accuracy. The implementation leverages TensorFlow and the `transformers` library, aligning with the practical approaches in *Advanced Natural Language Processing with TensorFlow 2*.

Training and Evaluation

- Pretraining: GPT models are pretrained on large, diverse corpora with CLM, requiring significant computational resources (e.g., GPT-3's training cost millions of dollars).
- Fine-Tuning: For sentiment analysis, fine-tune on labeled datasets (e.g., IMDB) with cross-entropy loss, though GPT-3 often relies on few-shot prompting.
- Evaluation:
  - Metrics: Accuracy, precision, recall, F1 score.
  - Example (GPT-2, hypothetical on IMDB):
    - Input: "Great acting, but slow plot."
    - Predicted: Negative.

- Metrics: Accuracy = 80% (lower than BERT's ~90% due to unidirectional context).
- Benchmarks: Compare against BERT (SST-2: ~93%) or RoBERTa (~95%) to highlight trade-offs.

Advantages of GPT Models

- Generative Power: Excel in text generation, making them versatile for tasks beyond classification (e.g., sentiment-aware dialogue).
- Few-Shot Learning: GPT-3's ability to perform tasks with minimal examples reduces fine-tuning needs.
- Scalability: Larger models (GPT-3) handle diverse tasks with high performance.
- Flexibility: Adaptable to creative and interactive applications, unlike BERT's focus on understanding.

Limitations

- Unidirectional Context: Less effective for tasks requiring full sequence understanding (e.g., sentiment analysis compared to BERT).
- Computational Cost: Training and inference for GPT-3 are resource-intensive, limiting accessibility.
- Bias and Ethics: Large-scale web data introduces biases, and generative capabilities risk misuse (e.g., misinformation).
- Overfitting: Fine-tuning on small datasets can degrade general knowledge.

Real-World Applications

- Sentiment Analysis: GPT-3 for few-shot classification in customer feedback or social media monitoring.
- Chatbots: Power conversational agents like ChatGPT, responding with sentiment awareness.
- Content Creation: Generate sentiment-aligned marketing copy or reviews.
- Code Generation: GPT-3 assists developers, indirectly supporting NLP tool development.

2. Applications of GPT Models

GPT models, particularly GPT-2 and GPT-3, are versatile, supporting applications in text generation, understanding, and even non-NLP domains. Below are prominent use cases:

1. Text Generation:
   - Description: GPT models generate coherent, contextually relevant text, such as articles, stories, or poetry, based on prompts.
   - Use Cases:
     - Content Creation: Blog posts, marketing copy, or social media content (e.g., generating a tweet: "Prompt: Write a tweet about AI advancements → 'AI is revolutionizing healthcare with faster diagnostics!'").
     - Creative Writing: Story continuation or scriptwriting (e.g., extending "Once upon a time" into a fairy tale).
   - Relevance to Sentiment Analysis: Generate sentiment-aligned text (e.g., positive reviews: "This movie was fantastic!") for data augmentation or marketing.
   - Example: GPT-3 powers tools like Jasper.ai for automated content creation.

2. Chatbots and Conversational AI:
   - Description: GPT models enable natural, context-aware dialogue, maintaining conversation coherence over multiple turns.
   - Use Cases:

- Customer Support: Virtual assistants (e.g., answering “How do I reset my password?” with step-by-step guidance).
  - Personal Assistants: Scheduling or answering queries (e.g., “What’s the weather today?”).
- Relevance to Sentiment Analysis: Detect user sentiment in chats (e.g., “I’m frustrated with this product”) and respond empathetically (“I’m sorry to hear that, let’s resolve this!”).
- Example: Early versions of ChatGPT, built on GPT-3, power conversational platforms.

3. Sentiment Analysis and Text Classification:
- Description: While unidirectional, GPT models can perform sentiment analysis via fine-tuning or few-shot prompting, classifying text as positive, negative, or neutral.
- Use Cases:
  - Social Media Monitoring: Analyze tweets for brand sentiment (e.g., “Love this new phone!” → Positive).
  - Customer Feedback: Classify reviews on e-commerce platforms.
- Relevance to Sentiment Analysis Case Study: GPT-3’s few-shot learning allows sentiment classification with prompts like “Classify: ‘Great movie!’ → Positive,” reducing the need for fine-tuning compared to BERT.
- Example: GPT-3 APIs are used in sentiment analysis tools for real-time feedback processing.

4. Machine Translation:
- Description: GPT models translate text between languages by generating translations based on prompts.
- Use Cases:
  - Multilingual Support: Translate customer queries in global e-commerce (e.g., “Bonjour, comment ça va?” → “Hello, how are you?”).
  - Content Localization: Adapt marketing materials across languages.
- Relevance to Sentiment Analysis: Translate sentiment-laden text while preserving emotional tone (e.g., “This film is amazing!” in Spanish).
- Example: GPT-3 supports translation in tools like DeepL’s experimental features.

5. Text Summarization:
- Description: GPT models condense long texts into concise summaries, preserving key information.
- Use Cases:
  - News Aggregation: Summarize articles for quick consumption (e.g., a 500-word article into 50 words).
  - Document Analysis: Summarize reports or legal documents.
- Relevance to Sentiment Analysis: Generate sentiment-focused summaries (e.g., “Customers love the product’s design but criticize its price”).
- Example: Tools like SummarizeBot leverage GPT-like models for summarization.

6. Code Generation and Assistance:
- Description: GPT models generate code snippets or assist developers by completing code based on natural language prompts.
- Use Cases:
  - Programming Support: Generate Python functions (e.g., “Write a function to sort a list” → `def sort_list(lst): return sorted(lst)`).
  - Debugging: Suggest fixes for code errors.
- Relevance to Sentiment Analysis: Generate scripts for sentiment analysis pipelines, enhancing development efficiency.
- Example: GitHub Copilot, powered by GPT-3-like models, assists coders in real-time.

7. Question Answering and Knowledge Retrieval:
- Description: GPT models answer questions by generating responses or extracting information from prompts or context.
- Use Cases:
  - Educational Tools: Answer student queries (e.g., "What is photosynthesis?").
  - Customer Support: Respond to FAQs (e.g., "What's your return policy?").
- Relevance to Sentiment Analysis: Answer sentiment-related queries (e.g., "Is this review positive?" → "Yes, it praises the product").
- Example: GPT-3 powers QA features in virtual assistants.

8. Educational Applications:
- Description: GPT models support learning by generating explanations, quizzes, or tutoring content.
- Use Cases:
  - Language Learning: Generate practice dialogues or grammar exercises.
  - Study Aids: Create summaries or flashcards from textbooks.
- Relevance to Sentiment Analysis: Generate sentiment-focused educational content (e.g., "Write a positive review for practice").
- Example: Duolingo's chatbot features use GPT-like models for interactive learning.

9. Creative and Entertainment Applications:
- Description: GPT models create artistic content, such as poems, music lyrics, or game narratives.
- Use Cases:
  - Game Development: Generate NPC dialogue or storylines.
  - Music and Literature: Write lyrics or short stories.
- Relevance to Sentiment Analysis: Create sentiment-aligned creative content (e.g., positive game narratives).
- Example: AI Dungeon uses GPT for interactive storytelling.

10. Business and Productivity Tools:
- Description: GPT models automate tasks like drafting emails, reports, or presentations.
- Use Cases:
  - Email Automation: Draft professional responses (e.g., "Thank you for your feedback").
  - Report Generation: Create business summaries from data.
- Relevance to Sentiment Analysis: Draft sentiment-appropriate responses (e.g., empathetic replies to negative feedback).
- Example: Grammarly's tone suggestions and Jasper.ai leverage GPT for productivity.

3. Text completion, summarization, and dialogue systems.

1. Text Completion

Description:
Text completion involves generating coherent continuations of a given text prompt, predicting subsequent tokens based on context. GPT models, trained on causal language modeling (CLM), excel at this by modeling the probability of the next token given previous ones.

Mechanism:
- Input: A partial text sequence (e.g., "The future of AI is").
- Processing: The GPT decoder uses masked self-attention to process tokens sequentially, predicting the next token's probability distribution.

- Output: A completed sequence (e.g., “The future of AI is bright and transformative”).
- Techniques: Sampling (e.g., top-k, top-p) or beam search ensures diverse, high-quality outputs.

Use Cases:
- Content Creation: Autocomplete emails or articles (e.g., drafting “Dear Customer, thank you for…” into a full response).
- Code Completion: Suggest code snippets (e.g., “def sort_list(” → “def sort_list(lst): return sorted(lst)”).
- Creative Writing: Extend story prompts (e.g., “Once upon a time” into a narrative).
- Relevance to Sentiment Analysis: Generate sentiment-aligned completions (e.g., positive continuations for “This movie is…”).

Real-World Examples:
- GitHub Copilot uses GPT-like models for code autocompletion.
- Google’s Smart Compose in Gmail suggests email continuations.

Challenges:
- Coherence: Long completions may drift off-topic.
- Bias: Outputs may reflect training data biases (e.g., overly optimistic completions).
- Latency: Real-time completion requires optimized models.

2. Text Summarization

Description:
Text summarization condenses long documents into concise summaries, preserving key information. GPT models perform abstractive summarization, generating new text rather than extracting sentences, leveraging their generative capabilities.

Mechanism:
- Input: A document or article (e.g., a 500-word news article).
- Processing: The GPT model processes the input, using self-attention to identify salient points, and generates a summary via autoregressive decoding.
- Output: A short summary (e.g., 50 words summarizing the article’s main points).
- Techniques: Prompt engineering (e.g., “Summarize this article:”) or fine-tuning on summarization datasets enhances quality.

Use Cases:
- News Aggregation: Summarize articles for quick consumption (e.g., summarizing a report on AI advancements).
- Business Reports: Condense financial or project reports for executives.
- Academic Research: Summarize papers for literature reviews.
- Relevance to Sentiment Analysis: Generate sentiment-focused summaries (e.g., “Customers praise the product’s design but note high costs”).

Real-World Examples:
- Tools like SummarizeBot or QuillBot use GPT-like models for summarization.
- News apps (e.g., Google News) provide article summaries.

Challenges:
- Information Loss: Key details may be omitted in short summaries.
- Hallucination: Generated summaries may include unverified information.
- Scalability: Summarizing long documents requires large context windows.

3. Dialogue Systems

Description:
Dialogue systems enable natural, context-aware conversations, maintaining coherence across multiple turns. GPT models power chatbots and virtual assistants by generating responses based on conversational context.

Mechanism:
- Input: A conversation history (e.g., “User: How’s the weather? Assistant: It’s sunny!”).
- Processing: The model encodes the dialogue, using self-attention to track context, and generates a response token-by-token.
- Output: A contextually appropriate response (e.g., “User: Great, any plans? Assistant: Maybe a picnic!”).
- Techniques: Prompt engineering, fine-tuning on dialogue datasets, or reinforcement learning (e.g., RLHF in ChatGPT) improves response quality.

Use Cases:
- Customer Support: Handle queries (e.g., “How do I return this item?”).
- Virtual Assistants: Schedule tasks or answer questions (e.g., “Set a reminder for 3 PM”).
- Entertainment: Power interactive chatbots in games or storytelling apps.
- Relevance to Sentiment Analysis: Detect user sentiment (e.g., “I’m upset”) and respond empathetically, enhancing user experience.

Real-World Examples:
- ChatGPT, built on GPT-3, powers conversational AI.
- Google’s Dialogflow and Microsoft’s Bot Framework integrate GPT-like models.

Benefits

- Text Completion:
  - Enhances productivity (e.g., email autocompletion).
  - Supports creative tasks with coherent outputs.
- Text Summarization:
  - Saves time by condensing information.
  - Enables scalable content processing (e.g., news feeds).
- Dialogue Systems:
  - Improves user engagement with natural responses.
  - Automates customer support, reducing costs.

Relevance to Sentiment Analysis:
- Generate sentiment-aligned completions (e.g., positive reviews).
- Summarize sentiment in feedback (e.g., “Most reviews are positive”).
- Respond to user sentiment in dialogues (e.g., empathetic replies).

Challenges

- Coherence and Accuracy:
  - Text completion may produce off-topic outputs.
  - Summaries risk omitting key details or hallucinating facts.
  - Dialogue systems may lose context in long conversations.

- Computational Cost: Large models (e.g., GPT-3) require significant resources, limiting real-time deployment (as of May 16, 2025).
- Bias and Safety: Outputs may reflect biases or generate inappropriate content, requiring moderation.
- Evaluation Complexity: Assessing quality (e.g., coherence, factual accuracy) often requires human judgment.

Evaluation

- Text Completion:
  - Metrics: BLEU, perplexity, human evaluation (coherence, relevance).
  - Example: Prompt: "AI is" → Output: "AI is transforming industries." (Human score: 4/5).
- Text Summarization:
  - Metrics: ROUGE (n-gram overlap), human evaluation (informativeness).
  - Example: Summary of a 500-word article in 50 words, ROUGE-1 = 0.6.
- Dialogue Systems:
  - Metrics: BLEU, dialogue coherence, user satisfaction.
  - Example: Response to "I'm upset" → "I'm sorry, let's fix this!" (Satisfaction: 90%).
- Benchmarks: Compare against BART (summarization) or BERT (classification) on datasets like CNN/DailyMail, MultiWOZ, or custom sentiment datasets.

4. Generating Text-one character at a time

Generating text one character at a time is a specialized approach in natural language processing (NLP) where a model predicts the next character in a sequence, rather than tokens or words, enabling fine-grained control over text generation. This method is particularly relevant for applications requiring precise output, such as text completion, code generation, or creative writing, and can be implemented using Generative Pre-trained Transformer (GPT) models.

Mechanism of Character-Level Text Generation

- In character-level generation, the model treats each character (e.g., letters, numbers, punctuation) as a token, predicting the next character based on the preceding sequence.
- Unlike word or subword-based models (e.g., BERT's WordPiece, GPT's BPE), character-level models operate on raw text, requiring no predefined vocabulary beyond the character set (e.g., a-z, A-Z, 0-9, punctuation).

Process:

1. Input: A sequence of characters (e.g., "Hell").
2. Encoding: Each character is mapped to a unique ID using a character vocabulary (e.g., 'H' → 0, 'e' → 1).
3. Model: A Transformer decoder (e.g., GPT-like) processes the sequence, using masked self-attention to model dependencies and predict the next character's probability distribution.
4. Output: The model samples the next character (e.g., 'o' for "Hello"), appending it to the sequence, and repeats until a stopping condition (e.g., max length, end token).
5. Decoding: The sequence of character IDs is converted back to text.

Self-Attention:

- The Transformer uses multi-head self-attention to compute relationships between characters:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where Q , K , and V are query, key, and value vectors, and d_k is the head dimension.

- Masked attention ensures the model only attends to previous characters, maintaining autoregressive generation.

Advantages:
- Flexibility: Handles arbitrary text, including misspellings, code, or non-standard words, without vocabulary constraints.
- Granularity: Enables precise control, ideal for tasks like spell-checking or code generation.
- Compact Vocabulary: Character sets (e.g., ~100 characters) are smaller than word/subword vocabularies (~30,000 for GPT).

Challenges:
- Longer Sequences: Character-level models require longer sequences than word-based models (e.g., "hello" is 5 tokens vs. 1), increasing computational cost.
- Coherence: Maintaining long-range dependencies is harder due to fine-grained modeling.
- Training Data: Requires large, diverse text corpora to learn character patterns effectively.

Use Cases

1. Text Completion:
   - Autocomplete text in editors or messaging apps (e.g., "Th" → "The quick brown fox").
   - Relevance to Sentiment Analysis: Generate sentiment-aligned completions (e.g., "This mov" → "This movie is amazing" for positive sentiment).
2. Code Generation:
   - Generate code character-by-character, capturing syntax precisely (e.g., "def f" → "def factorial(n):").
   - Useful for tools like GitHub Copilot, supporting programming languages with unique syntax.
3. Spell-Checking and Correction:
   - Detect and correct errors by predicting correct characters (e.g., "recieve" → "receive").
   - Ties to the sentiment analysis case study by enhancing sentiment-aware text preprocessing.
4. Creative Writing:
   - Generate poetry or stories with fine control over style (e.g., "In a f" → "In a forest dark and deep").
   - Supports sentiment-aligned narratives (e.g., positive or negative tones).
5. Dialogue Systems:
   - Generate conversational responses character-by-character for real-time applications.
   - Enhances sentiment-aware chatbots by ensuring precise, contextually appropriate outputs.

Benefits
- Precision: Character-level generation ensures exact control, ideal for code or spell-checking.
- Flexibility: Handles any text, including non-standard words or multilingual scripts.
- Sentiment Alignment: Can generate sentiment-specific text (e.g., positive completions for reviews), tying to the sentiment analysis case study.
- Compact Model: Smaller vocabulary reduces memory requirements compared to subword models.

Challenges
- Computational Cost: Longer sequences increase processing time and memory usage.
- Coherence: Maintaining long-term context is difficult, risking incoherent outputs.
- Training Data: Requires extensive text to learn diverse character patterns.
- Evaluation: Assessing quality (e.g., coherence, correctness) often needs human judgment.

5. Generative Pretraining (GPT-2) model

The Generative Pre-trained Transformer 2 (GPT-2) model, developed by OpenAI and released in 2019, is a landmark in natural language processing (NLP), building on the foundation of GPT-1 and paving the way for GPT-3. As a decoder-only Transformer, GPT-2 excels in generating coherent, contextually relevant text and supports tasks like text completion, summarization, and sentiment analysis.

Architecture of GPT-2

GPT-2 is a decoder-only Transformer, designed for autoregressive language modeling, predicting the next token in a sequence given previous tokens. Its architecture scales up from GPT-1, with multiple model sizes:

- Model Variants:
  - Small: 117M parameters (similar to GPT-1).
  - Medium: 345M parameters.
  - Large: 774M parameters.
  - XL: 1.5B parameters (largest at release).
- Layers: Up to 48 layers in the 1.5B model, compared to GPT-1's 12.
- Embedding Size: 1,600 dimensions (1.5B model) vs. GPT-1's 768.
- Context Window: 1,024 tokens, doubled from GPT-1's 512, allowing longer-range dependencies.
- Vocabulary: ~50,257 subword tokens using Byte Pair Encoding (BPE), balancing efficiency and flexibility.

Key Components:

- Multi-Head Self-Attention: Computes relationships between tokens, with masking to ensure unidirectional (left-to-right) processing:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where Q, K, and V are query, key, and value vectors, and d_k is the head dimension (e.g., 100 for 1,600 dimensions, 16 heads).

- Feed-Forward Neural Network (FFN): Position-wise dense layers with ReLU activation.
- Positional Encoding: Sinusoidal encodings to capture token order, critical for sequence modeling.
- Layer Normalization: Applied at sub-block inputs (unlike GPT-1's output normalization), improving training stability.
- Residual Connections: Enhance gradient flow across deep layers.

Scalability:

- GPT-2's architecture supports scaling by increasing layers, embedding size, and heads, demonstrating that larger models improve performance, a trend continued in GPT-3.

Pretraining Strategy

GPT-2 is pretrained using an unsupervised approach to learn general language representations from a large, diverse corpus.

- Objective: Causal Language Modeling (CLM):
  - Predict the next token given prior tokens, maximizing the likelihood:

$$L = -\sum_t \log P(x_t | x_{1:t-1})$$

where x_t is the t-th token.

  - Autoregressive nature ensures generation aligns with human-like text production.
- Dataset: WebText:
  - Curated from ~8M web pages linked on Reddit (with at least 3 upvotes), yielding ~40GB of text.
  - Diverse content: blogs, forums, news, compared to GPT-1's BookCorpus (~800M words).
  - Filtered for quality, removing low-value content (e.g., boilerplate text).
- Training:
  - Unsupervised, using gradient-based optimization (e.g., Adam).
  - Large-scale compute (e.g., GPUs/TPUs), with the 1.5B model requiring significant resources.

- No explicit task-specific data, enabling broad applicability.

Key Innovation:

- Emphasis on zero-shot learning: GPT-2 generates coherent text or performs tasks without fine-tuning, relying on patterns learned during pretraining.

Key Features

1. Zero-Shot Performance:
   - Unlike GPT-1's reliance on fine-tuning, GPT-2 performs tasks like text generation, translation, or question answering using prompts alone (e.g., "Translate 'Hello' to French" → "Bonjour").
   - Precursor to GPT-3's few-shot learning.
2. Improved Text Quality:
   - Generates longer, more coherent sequences than GPT-1, handling paragraphs or stories.
   - Maintains context over 1,024 tokens, improving narrative consistency.
3. Scalability:
   - Larger models (e.g., 1.5B parameters) outperform smaller ones, demonstrating the benefits of scale.
4. Ethical Considerations:
   - OpenAI initially withheld the full 1.5B model due to misuse risks (e.g., generating fake news), releasing it later with safeguards.
   - Sparked discussions on responsible AI deployment.

Applications

GPT-2's generative capabilities support diverse applications, many of which tie to the sentiment analysis case study and prior discussions:

1. Text Generation:
   - Generate articles, stories, or marketing copy (e.g., "Write a blog about AI" → coherent post).
   - Sentiment Analysis Tie-In: Generate sentiment-aligned text (e.g., positive reviews: "This movie is spectacular!").
2. Text Completion:
   - Autocomplete prompts (e.g., "The future of AI" → "is transformative and ethical").
   - Supports code completion (e.g., "def f" → Python function).
3. Summarization:
   - Condense articles or reports (e.g., summarize a 500-word news article into 50 words).
   - Can focus on sentiment (e.g., "Reviews praise the product's design").
4. Dialogue Systems:
   - Power chatbots or virtual assistants (e.g., "User: How's the weather? Assistant: Sunny!").
   - Detect and respond to user sentiment (e.g., empathetic replies to negative feedback).
5. Sentiment Analysis:
   - Perform zero-shot classification with prompts (e.g., "Is 'Great movie!' positive?" → "Yes").
   - Less accurate than BERT but flexible for few-shot setups.
6. Translation and Question Answering:
   - Translate text (e.g., "Hola" → "Hello") or answer queries (e.g., "What is AI?").
   - Enhances sentiment-aware multilingual applications.

Real-World Examples:

- Writing Tools: Jasper.ai uses GPT-like models for content creation.
- Chatbots: Early conversational AI platforms leveraged GPT-2.
- Research: GPT-2 powers experiments in creative writing and NLP benchmarks.

Limitations

1. Unidirectional Context:
   - Left-to-right processing limits performance on tasks requiring full sequence understanding (e.g., sentiment analysis vs. BERT's bidirectional context).
2. Computational Cost:
   - The 1.5B model requires significant GPU/TPU resources for training and inference, challenging for smaller organizations (as of May 20, 2025).

3. Bias and Safety:
   - WebText includes biases (e.g., skewed perspectives from Reddit), risking biased outputs.
   - Potential for generating harmful content (e.g., misinformation) necessitates moderation.
4. Coherence Over Long Sequences:
   - May lose context or produce repetitive text in extended generations.
5. Fine-Tuning Dependency:
   - While zero-shot capable, fine-tuning improves performance for specific tasks, requiring labeled data.

Relevance to Prior Discussions

- Sentiment Analysis Case Study:
  - GPT-2 can perform zero-shot sentiment classification (e.g., "Classify: 'Loved it!' → Positive") or generate sentiment-aligned text, complementing BERT's classification focus.
  - Less accurate than RoBERTa (~94.8% on SST-2 vs. GPT-2's ~80–85%) but more flexible for generative tasks.
- Text Completion and Dialogue:
  - Supports character-level generation (e.g., "Th" → "The") or dialogue responses, as explored in prior responses.
- Other Pretrained Models:
  - Compared to T5 (encoder-decoder) or BERT (encoder-only), GPT-2's decoder-only design prioritizes generation over understanding, making it ideal for creative applications.

Implementation: GPT-2 for Text Generation

```
import tensorflow as tf
from transformers import TFGPT2LMHeadModel, GPT2Tokenizer

# Initialize model and tokenizer
model_name = 'gpt2-medium'  # Using medium for balance of performance and resources
tokenizer = GPT2Tokenizer.from_pretrained(model_name)
model = TFGPT2LMHeadModel.from_pretrained(model_name)

# Function to generate sentiment-aligned text
def generate_text(prompt, sentiment='Positive', model=model, tokenizer=tokenizer, max_len=100, max_new_tokens=50):
    # Incorporate sentiment into prompt
    full_prompt = f"Write a {sentiment.lower()} review: {prompt}"

    # Tokenize input
    encoded = tokenizer(full_prompt, max_length=max_len, truncation=True, return_tensors='tf')
    input_ids = encoded['input_ids']
    attention_mask = encoded['attention_mask']

    # Generate text
    outputs = model.generate(
        input_ids=input_ids,
        attention_mask=attention_mask,
        max_new_tokens=max_new_tokens,
        do_sample=True,  # Enable sampling for diversity
        top_k=50,        # Top-k sampling
        top_p=0.9,       # Top-p (nucleus) sampling
        pad_token_id=tokenizer.eos_token_id
    )
```

```
    # Decode output
    generated_text = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return generated_text

# Example usage
# Positive sentiment
prompt_positive = "This movie has great visuals"
generated_positive = generate_text(prompt_positive, sentiment='Positive')
print(f"Positive Prompt: {prompt_positive}")
print(f"Generated: {generated_positive}\n")

# Negative sentiment
prompt_negative = "The movie was slow"
generated_negative = generate_text(prompt_negative, sentiment='Negative')
print(f"Negative Prompt: {prompt_negative}")
print(f"Generated: {generated_negative}")
```

This code uses GPT-2 (medium variant, 345M parameters) to generate sentiment-aligned text, demonstrating its generative capabilities. For production, fine-tune on task-specific data (e.g., IMDB for sentiment) or use GPT-3 via API for enhanced quality. The implementation aligns with *Advanced Natural Language Processing with TensorFlow 2*'s emphasis on practical NLP solutions.

Evaluation

- Metrics:
  - Perplexity: Measures prediction uncertainty (lower indicates better modeling).
  - BLEU: Assesses similarity to reference texts.
  - Human Evaluation: Scores coherence, relevance, and sentiment alignment (1–5 scale).
- Example (hypothetical):
  - Prompt: "This movie has great visuals"
  - Output: "This movie has great visuals and a captivating story, a must-watch!"
  - Perplexity: 15, BLEU: 0.7, Human Score: 4.5/5 (positive sentiment aligned).
- Benchmarks:
  - Compare against GPT-1 (weaker coherence) or BART (better for summarization) on datasets like WebText or IMDB.
  - Sentiment accuracy: ~80–85% in zero-shot settings vs. BERT's ~90% (SST-2).

Real-World Impact

- Commercial Tools: GPT-2 powers early versions of writing assistants (e.g., WriteSonic) and chatbots.
- Research: Drives studies in generative AI, zero-shot learning, and ethical AI.
- Industry: Supports automation in content creation, customer service, and education.
- Sentiment Analysis: Enhances sentiment-aware tools by generating contextually appropriate text.

6. Text Summarization with seq2seq Attention and Transformer Networks.

Text summarization is a critical natural language processing (NLP) task that involves condensing a long document or piece of text into a concise summary while preserving its key information and meaning. Sequence-to-sequence (seq2seq) models, enhanced with attention mechanisms and Transformer networks, have become powerful tools for abstractive text summarization, where the model generates new text rather than extracting existing sentences.

Types of Summarization:
- Extractive: Selects key sentences or phrases from the input text (e.g., ranking sentences by importance).
- Abstractive: Generates new text that paraphrases the input, capturing its essence (e.g., rewriting a 500-word article into a 50-word summary).
Seq2Seq Models:
- Seq2seq models map an input sequence (source text) to an output sequence (summary), making them ideal for abstractive summarization.
- Traditional seq2seq models used recurrent neural networks (RNNs), but attention mechanisms and Transformers have largely replaced RNNs due to their efficiency and performance.

Attention and Transformers:

- **Attention:** Allows the model to focus on relevant parts of the input when generating each output token, addressing the limitations of fixed-length context vectors in RNN-based seq2seq models.
- Transformers: Use self-attention and cross-attention to process sequences in parallel, capturing long-range dependencies and scaling effectively.
Seq2Seq with Attention for Text Summarization

Architecture:
- Encoder: Processes the input text, producing a set of hidden states representing the source sequence.
  - Typically an RNN (e.g., LSTM, GRU) or a Transformer encoder.
  - For RNNs, the encoder outputs a sequence of hidden states h_1, h_2, \ldots, h_n and a final context vector.
- Decoder: Generates the summary, predicting one token at a time.
  - Also an RNN or Transformer decoder, conditioned on the encoder's outputs.
- Attention Mechanism:
  - Introduced by Bahdanau et al. (2015), attention computes a weighted sum of encoder hidden states for each decoder step, focusing on relevant input tokens.
  - Attention score for input token i at decoder step t :

$$\text{score}(h_i, s_t) = v_a^\top \tanh(W_a[h_i; s_t])$$

where h_i is the encoder hidden state, s_t is the decoder state, and v_a, W_a are learned parameters.
  - Attention weights:

$$\alpha_{ti} = \frac{\exp(\text{score}(h_i, s_t))}{\sum_j \exp(\text{score}(h_j, s_t))}$$

  - Context vector:

$$c_t = \sum_i \alpha_{ti} h_i$$

  - The decoder uses c_t alongside its state to predict the next token.
Limitations of RNN-based Seq2Seq with Attention:
- Sequential Processing: RNNs process tokens sequentially, limiting scalability for long texts.
- Vanishing Gradients: Long sequences degrade performance despite attention.
- Memory Constraints: Fixed context vectors struggle with very long inputs.

Transformer Networks for Text Summarization

Architecture:

Transformers, introduced by Vaswani et al. (2017) in *Attention is All You Need*, replace RNNs with self-attention, enabling parallel processing and better handling of long-range dependencies. For summarization, a Transformer-based seq2seq model consists of:

- **Encoder:**
  - Stack of N layers (e.g., 6–12), each with:
    - Multi-Head Self-Attention: Computes relationships between all input tokens:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

where Q, K, V are query, key, and value matrices, and d_k is the head dimension.

    - Feed-Forward Neural Network (FFN): Position-wise dense layers with ReLU.
    - Residual Connections and Layer Normalization: Stabilize training.
  - Input: Tokenized text with embeddings and positional encodings (to capture order).
  - Output: Contextualized representations of the input sequence.
- **Decoder:**
  - Stack of N layers, each with:
    - Masked Multi-Head Self-Attention: Ensures autoregressive generation by masking future tokens.
    - Multi-Head Cross-Attention: Attends to encoder outputs, aligning input and output sequences.
    - FFN, Residual Connections, and Layer Normalization.
  - Output: Probability distribution over the vocabulary for each token in the summary.
- Positional Encoding:
  - Sinusoidal or learned encodings to represent token positions, as Transformers lack sequential processing.

Advantages Over RNNs:

- Parallelization: Processes all tokens simultaneously, speeding up training and inference.
- Long-Range Dependencies: Self-attention captures relationships across long sequences (e.g., 512–2,048 tokens).
- Scalability: Handles large datasets and models (e.g., millions of parameters).

Key Models:

- T5 (Text-to-Text Transfer Transformer): Treats summarization as a text-to-text task, pretrained with span corruption.
- BART (Bidirectional and Auto-Regressive Transformer): Combines bidirectional encoding with autoregressive decoding, optimized for summarization.
- PEGASUS: Specifically designed for summarization, pretrained with a gap-sentence generation objective.

Training Strategy

Pretraining:

- Objective: Learn general language representations using unsupervised tasks.
  - T5: Span corruption (predict masked spans of text).
  - BART: Text infilling (mask and reconstruct spans) and sentence shuffling.
  - PEGASUS: Gap-sentence generation (mask key sentences, predict them).
- Dataset: Large corpora like C4 (Colossal Clean Crawled Corpus), Wikipedia, or Common Crawl.
- Compute: Requires GPUs/TPUs for training on billions of tokens.

Fine-Tuning:

- Objective: Adapt the model to summarization using supervised data.
  - Loss: Cross-entropy loss, comparing predicted tokens to reference summaries.
  - Dataset: Specialized datasets like:
    - CNN/DailyMail: News articles with human-written summaries.
    - XSum: Extreme summarization with one-sentence summaries.
    - BIGPATENT: Patent documents with abstracts.
  - Techniques: Learning rate scheduling (e.g., 1e-4), early stopping, and beam search for decoding.
- Hyperparameters: Small learning rates (e.g., 1e-5 for Transformers), 2–5 epochs, batch size 8–32.

Data Augmentation:
- Back-translation or synthetic data generation to improve robustness.
- Sentiment-aware fine-tuning (e.g., summarizing positive vs. negative reviews), tying to the sentiment analysis case study.

Applications

1. News Aggregation:
   - Summarize articles for quick consumption (e.g., a 500-word article into a 50-word summary).
   - Example: Summarize a report on AI advancements for a news app.
2. Business and Productivity:
   - Condense reports, emails, or meeting notes for executives.
   - Example: Summarize a 10-page financial report into key points.
3. Academic Research:
   - Summarize papers or literature reviews for researchers.
   - Example: Condense a 20-page study into a one-paragraph abstract.
4. Customer Feedback Analysis:
   - Summarize reviews or survey responses, focusing on sentiment.
   - Relevance to Sentiment Analysis: Generate summaries like "Customers praise the product's design but criticize its price," aligning with the sentiment analysis case study.
5. Legal and Medical Domains:
   - Summarize contracts, case law, or medical records for professionals.
   Example: Summarize a patient's chart for quick review.

Benefits
- Abstractive Summaries: Generates new, concise text, improving readability over extractive methods.
- Context Awareness: Attention and Transformers capture long-range dependencies, ensuring relevant summaries.
- Scalability: Parallel processing handles large documents efficiently.
- Sentiment Alignment: Can prioritize sentiment (e.g., positive aspects), tying to the sentiment analysis case study.

Challenges

- Hallucination: Models may generate unverified or incorrect information.
- Computational Cost: Training and inference require significant resources (GPUs/TPUs), especially for large models like T5-11B.
- Evaluation Complexity: Metrics like ROUGE measure n-gram overlap but may not capture semantic quality, requiring human evaluation.
- Data Dependence: High-quality summaries need large, diverse datasets.